# Small Molecule Design Using a Causal Transformer on SMILES Representations

Shubham Kumar, Sneha Yadav, Sparsh Varshney, Shubham Kumar, Shiwani Kumari Panjiya
Dept. of Computer Science & Engineering (AI/ML)
IILM University, Greater Noida, India
kumar.shubham.cs27@iilm.edu, sneha.yadav.cs27@iilm.edu, sparsh.varshney.cs27@iilm.edu,
shubham1.kumar.cs27@iilm.edu , shiwani.kumari.cs27@iilm.edu

***Abstract*—Navigating the vast space of drug-like molecules—estimated at $10^{60}$ candidates—is a central bottleneck in early-stage drug discovery. We present MolecularTransformer, a compact 3.2 M-parameter causal Transformer that learns to generate novel small molecules by autoregressively predicting SMILES tokens. Trained on 249,455 canonicalized strings from ZINC-250k with scaffold-aware Murcko splitting (80/10/10), the model uses a regex-based 65-token vocabulary, 256-dimensional embeddings, 8-head self-attention across four Pre-LN encoder layers, and an OneCycleLR-scheduled AdamW optimizer with label smoothing. A multi-stage decoding pipeline—temperature scaling, Top-*k*/Top-*p* nucleus sampling, and repetition penalty—is validated by ablation to improve chemical validity by 6.2 percentage points over temperature-only sampling. At $T=0.8$, the system achieves 84.0% validity, mean QED of 0.744, mean SA score of 2.99, and 96.4% Lipinski Rule-of-Five compliance, with internal diversity comparable to larger baselines. An open-source Streamlit dashboard provides real-time 3D visualization, t-SNE chemical-space mapping, and Tanimoto similarity search. The deliberately compact architecture enables training on free-tier GPU hardware, lowering the barrier to AI-assisted molecular design. To the best of our knowledge, MolecularTransformer is among the few open-source systems that unify scaffold-aware splitting, ablation-validated multi-stage sampling, nine-descriptor evaluation, and interactive deployment within a single consumer-hardware-trainable pipeline.**

***Index Terms*—SMILES, Transformer, molecular generation, drug discovery, deep learning, cheminformatics**

## I. Introduction

Rational drug design begins with identifying candidate molecules that are structurally novel, chemically stable, and pharmacologically promising. The scale of this search is daunting: conservative estimates place the number of synthetically feasible drug-like compounds at $10^{60}$ [1], whereas even the most aggressive high-throughput screening (HTS) campaigns assess only a negligible fraction of that space. Coupled with development costs that now exceed $2.6 billion per approved drug [2] and timelines of 10–15 years, these realities have spurred sustained interest in generative models that can autonomously propose viable candidates without exhaustive enumeration.

Code and live demo: https://github.com/Shubham12864/Small-Molecule-Design-Using-Transformer-model and https://small-molecule-design-using-transformer-model-fzvu6ve66dz8h8zf.streamlit.app

A practical bridge between molecular graphs and sequence models is provided by the SMILES notation [3], which linearises a molecular graph into a character string encoding atoms, bonds, branches, and ring closures. For instance, the analgesic Aspirin is compactly written as `CC(=O)Oc1ccccc1C(=O)O`. Because SMILES strings obey a context-sensitive grammar, generating valid molecules can be cast as a sequential token-prediction task—opening the door to architectures originally developed for natural language. Among such architectures, the Transformer [4] is particularly well matched to SMILES. Its self-attention layers relate every pair of tokens in constant depth, which is critical for maintaining ring-closure consistency: two matching digits (e.g., the pair of `1`s in `c1ccccc1`) may be separated by many intervening tokens, a dependency that recurrent networks must thread through numerous hidden states. A causal (left-to-right) attention mask converts the Transformer into an autoregressive generator while retaining this global receptive field.

We present **MolecularTransformer**, an end-to-end open-source system for de novo molecular generation. Unlike prior work that treats model training, property evaluation, and deployment as disconnected stages, our principal contribution is a unified, reproducible pipeline—from scaffold-aware data preparation through ablation-validated multi-stage sampling to interactive web-based molecular exploration—packaged as a single compact system trainable on a free-tier GPU. To clearly scope our claims, we separate *system-level contributions* from *validated design choices* whose individual merits are confirmed experimentally rather than claimed as novel:

**System-level contributions:**

- A fully reproducible pipeline spanning data preprocessing, scaffold-aware partitioning via Murcko decomposition [5], model training, nine-metric RDKit evaluation, and one-click Streamlit deployment—all executable on consumer hardware.
- A nine-descriptor drug-likeness evaluation suite (MW, LogP, QED [6], SA [7], TPSA, HBD, HBA, rotatable bonds, ring count) augmented with a composite Lipinski Ro5 [8] compliance flag.
- An interactive web dashboard integrating 3D py3Dmol rendering, t-SNE [9] chemical-space projection (Morgan fingerprints, radius 2, 2048 bits), and Tanimoto-based

known-drug similarity search.

**Validated design choices:**

- A regex-based SMILES tokenizer with a 65-token vocabulary that correctly segments multi-character atoms (`Cl`, `Br`), bracketed stereo atoms (`[C@H]`), and `%NN` ring closures—eliminating errors caused by naive character-level splitting.
- A 3.2 M-parameter causally-masked encoder stack (Pre-LN, GELU, $d_{\text{model}}$=256, 8 heads) optimised with OneCycleLR-scheduled AdamW [10] and label smoothing ($\varepsilon$=0.05).
- A multi-stage decoding pipeline (temperature, Top-$k$, Top-$p$ [11], repetition penalty, minimum-length guard) whose cumulative benefit is quantified by ablation (Table IV).

## II. Related Work

### A. Recurrent and Variational Approaches

The earliest neural approaches to SMILES-based generation employed recurrent architectures. Segler et al. [12] showed that character-level LSTM networks, when exposed to sufficiently large molecular corpora, can internalise the syntactic rules governing SMILES and produce valid candidates at rates as high as 97.4% on ZINC-250k [13]. Pursuing a different objective, Go´mez-Bombarelli et al. [14] trained a variational autoencoder (VAE) [15] that projects SMILES into a smooth latent manifold, thereby enabling gradient-guided property optimisation; their reconstruction accuracy, however, was limited to 67.5% valid outputs. Both families of models share a structural weakness: RNNs propagate information through a chain of hidden-state updates whose effective memory degrades over long sequences [16], while VAEs force entire sequences through a fixed-capacity bottleneck, making it inherently difficult to preserve the non-local token correspondences (e.g., ring-closure digit pairs and nested branches) that SMILES validity demands.

### B. Transformer-Based Molecular Generation

Self-attention architectures address the long-range limitation by computing pairwise token interactions in a single layer. Bagal et al. [17] applied a GPT-style decoder Transformer to SMILES generation (MolGPT), achieving 88.9% validity and 96.5% uniqueness on ZINC-250k and demonstrating measurable gains over recurrent baselines. On the representation-learning front, Chithrananda et al. [18] pre-trained ChemBERTa, a masked-language model over SMILES, for downstream property-prediction tasks rather than generation. Orthogonally, the SELFIES encoding [19] guarantees syntactic correctness through valence-enforcing production rules, but sacrifices expressive flexibility and incurs additional decoding overhead.

Beyond sequential representations, recent work has explored three-dimensional generative modelling. Hoogeboom et al. [20] introduced equivariant diffusion processes that place atoms directly in Cartesian coordinates, while Irwin et al. [21] scaled up pre-training with Chemformer—a 100 M+ parameter Transformer trained on large reaction corpora. In contrast to these resource-intensive approaches, MolecularTransformer deliberately targets the opposite end of the scale spectrum: at 3.2 M parameters it is trainable on a single free-tier T4 GPU, making it accessible to researchers without dedicated compute infrastructure.

### C. Evaluation Protocols

Standardised comparison of molecular generators has been facilitated by the MOSES benchmarking suite [13], which prescribes validity, uniqueness, novelty, internal diversity (IntDiv), and Fre´chet ChemNet Distance (FCD) as core metrics evaluated on ZINC-250k. We report the subset of MOSES metrics computable without the proprietary ChemNet encoder (Table VI). Individual molecule quality is further assessed through Lipinski's Rule of Five [8], the quantitative estimate of drug-likeness (QED) [6], and the synthetic accessibility (SA) score [7].

## III. Dataset and Tokenization

### A. ZINC-250k Corpus

The training corpus is drawn from ZINC-250k [22], a widely used benchmark subset of the ZINC database containing 249,455 commercially available, drug-like compounds. Each SMILES string is first canonicalised with RDKit (`Chem.MolToSmiles`, canonical ordering) and duplicate entries are removed. The resulting set is split according to Murcko scaffold decomposition [5]: molecules sharing the same core ring-and-linker skeleton are assigned exclusively to one partition. The default configuration uses an 80/10/10 train/validation/test ratio. Because entire scaffold families reside in a single split, this scheme prevents structural leakage and provides a more rigorous generalisation test than random partitioning. A runtime check enforces zero inter-split overlap.

### B. Regex-Based SMILES Tokenizer

Naive character-level tokenisation of SMILES is chemically destructive: it splits two-character element symbols such as `Cl` and `Br` into separate, meaningless tokens. Our `SmilesTokenizer` instead applies a compiled regular expression (Listing 1) that recognises multi-character atoms, bracketed stereo/charged atoms (e.g., `[C@H]`, `[NH+]`), aromatic lowercase atoms, bond and branch operators, and two-digit ring closures (`%10`, `%11`).

```
SMILES_PATTERN = re.compile(
    r"(\[[^\]]+]|Br?|Cl?|N|O|S|P|F|I"
    r"|b|c|n|o|s|p"
    r"|\(|\)|\.|=|#|-|\+|\\|/|:|~|@"
    r"|\?|>|\*|\$|\%[0-9]{2}|[0-9])"
)
```

Listing 1. Regex pattern in `src/tokenizer.py` for chemically correct SMILES tokenisation.

The vocabulary derived from ZINC-250k contains 65 tokens, including four special symbols (`<PAD>`, `<SOS>`, `<EOS>`, `<UNK>`). This count is verified against the saved model checkpoint (`vocab_size: 65`).

## IV. Model Architecture

### A. Causally-Masked Encoder Rationale

MolecularTransformer is implemented with PyTorch's `nn.TransformerEncoderLayer` combined with a causal upper-triangular attention mask, making it functionally equivalent to a decoder-only Transformer such as MolGPT [17]. Two practical considerations motivate this choice: (i) the encoder API exposes `enable_nested_tensor=False`, which is required for stable mixed-precision training on variable-length SMILES, and (ii) the same stack can later be fine-tuned bidirectionally for property prediction without architectural modification. Because the causal mask enforces strictly left-to-right attention, generation behaviour is identical to that of a standard decoder.

### B. Architecture Details

The model (Listing 2) stacks four Pre-LayerNorm encoder layers with GELU activation. Tokenembeddings are drawn from $\mathcal{N}(0, 0.02)$ and scaled by $\sqrt{d_{\text{model}}}$ at forward time. The linear output head shares the same initialisation with zero-initialised biases; attention and feed-forward weights use Xavier uniform initialisation. Fig. 1 shows the end-to-end pipeline.

```python
class MolecularTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=256,
                 nhead=8, num_layers=4,
                 dim_feedforward=1024, max_len=512,
                 dropout=0.1, pad_token_id=0):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, d_model, padding_idx=pad_token_id
                )
        self.pos_encoder = PositionalEncoding(
            d_model, max_len, dropout)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout, activation="gelu",
            batch_first=True, norm_first=True)  # Pre-LN
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers,
            norm=nn.LayerNorm(d_model),
            enable_nested_tensor=False)
        self.output_head = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        causal_mask = self._generate_causal_mask(
            x.size(1), x.device)
        padding_mask = (x == self.pad_token_id)
        emb = self.embedding(x) * math.sqrt(self.d_model)
        emb = self.pos_encoder(self.emb_dropout(emb))
        out = self.transformer(emb,
            mask=causal_mask,
            src_key_padding_mask=padding_mask)
        return self.output_head(out)
```

Listing 2. MolecularTransformer core from `src/model.py` (checkpoint-verified).

### C. Causal Masking

Autoregressive generation is enforced through a causal mask:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \tag{1}$$

TABLE I
MolecularTransformer configuration (verified from `checkpoints/best_model.pt`).

| Parameter | Value |
|---|---|
| Embedding dimension ($d_{\text{model}}$) | 256 |
| Attention heads ($n_{\text{head}}$) | 8 |
| Encoder layers | 4 |
| Feed-forward dimension | 1024 |
| Activation function | GELU |
| Normalization | Pre-LN |
| Positional encoding | Sinusoidal (fixed) |
| Max sequence length | 60 tokens |
| Vocabulary size | 65 tokens |
| Dropout | 0.2 |
| Embedding scaling | $\times\sqrt{256}$ |
| **Total parameters** | **3,208,257** |

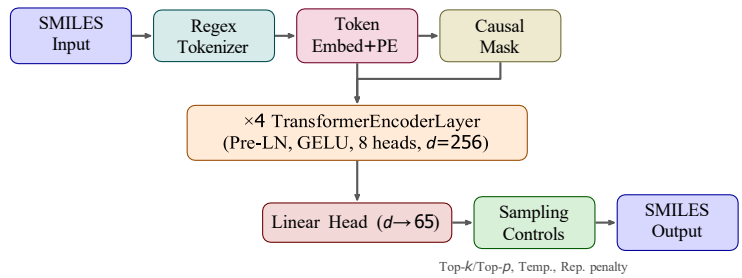


Fig. 1. End-to-end MolecularTransformer pipeline from SMILES input to generated output.

A separate padding mask zeroes attention weights at `<PAD>` positions, ensuring the model conditions only on valid preceding tokens.

### D. Positional Encoding

Fixed sinusoidal positional encodings [4] are added to token embeddings:

$$PE_{(pos,2i)} = \sin\frac{pos}{10000^{2i/d_{\text{model}}}} \tag{2}$$

$$PE_{(pos,2i+1)} = \cos\frac{pos}{10000^{2i/d_{\text{model}}}} \tag{3}$$

The positional buffer is sized to 60 tokens (verified from the checkpoint: `pe.shape = [1, 60, 256]`). At inference the generation loop reads this limit directly, preventing sequences from exceeding the trained positional context.

### E. Architecture Summary

Table I lists the complete, checkpoint-verified configuration. The total parameter count (3,208,257) is obtained by summing all state-dictionary tensors.

## V. Training

### A. Optimisation

Training uses AdamW [10] ($\lambda$=0.01) with a OneCycleLR schedule [23] (peak rate $3\times10^{-4}$, 10% linear warmup):

$$\eta(t) = \eta_{\max} \cdot f_{\text{OneCycle}}(t) \tag{4}$$

TABLE II
TRAINING HYPERPARAMETERS.

| Hyperparameter | Value |
|---|---|
| Optimizer | AdamW |
| Max learning rate | $3 \times 10^{-4}$ |
| LR schedule | OneCycleLR (10% warmup) |
| Weight decay | 0.01 |
| Label smoothing | 0.05 |
| Gradient clip norm | 1.0 |
| Batch size | 64 |
| Epochs (max) | 20 |
| Early stopping patience | 8 |
| Data split | Scaffold (Murcko), 80/10/10 |
| Deduplication | Yes |
| Mixed precision (AMP) | Yes (CUDA) |
| Hardware | NVIDIA T4 GPU |

where $f_{\text{OneCycle}}$ ramps linearly to $\eta_{\max}$ then decays via cosine annealing. Gradient norms are clipped at 1.0. Mixed-precision training (AMP with `GradScaler`) is enabled on CUDA devices.

### B. Loss Function

The objective is cross-entropy with label smoothing ($\varepsilon$=0.05), ignoring padding positions:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \tilde{y}_t \log P_\vartheta(x_t \mid x_{<t}) \tag{5}$$

with smoothed targets $\tilde{y}_t = (1-\varepsilon)\, y_t + \varepsilon/|V|$. Spreading a small residual probability across all vocabulary entries prevents the model from converging to degenerate, high-certainty distributions on frequent tokens.

### C. Training Configuration and Convergence

Table II summarises all hyperparameters. Training ran for 20 epochs on an NVIDIA T4 GPU; early stopping (patience 8, min_delta $10^{-4}$) did not trigger.

Fig. 2 plots training and validation loss across all 20 epochs (data from `checkpoints/loss_history.csv`). Training loss decreased monotonically from 1.771 (epoch 1) to 1.000 (epoch 20). Validation loss reached its minimum of 1.123 at epoch 16 and then plateaued, suggesting the model had saturated its representational capacity at this scale. The best checkpoint was saved at epoch 16.

## VI. MOLECULE GENERATION

### A. Autoregressive Sampling

Generation starts from the `<SOS>` token followed by an optional seed fragment (e.g., `"C"`, `"N"`, or `"c1ccccc1"`). At each time step, the model outputs a logit vector over the vocabulary; the next token is selected through a multi-stage filtering pipeline described below.

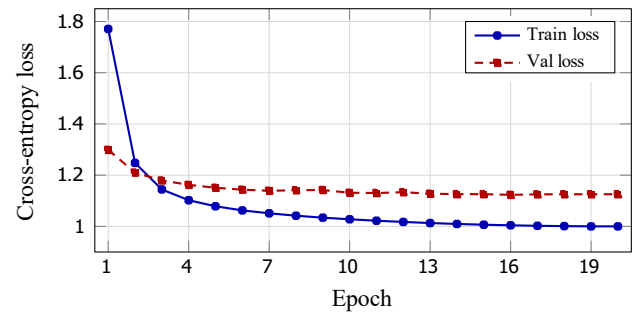


Fig. 2. Loss curves over 20 epochs. The best checkpoint is saved at epoch 16 (val. loss = 1.123).

### B. Generation Controls

Four layered controls shape the output distribution:

**Temperature scaling** adjusts distribution sharpness:

$$P(x_t = k) = \frac{\exp(l_k/T)}{\sum_j \exp(l_j/T)} \tag{6}$$

with $T \in [0.1, 1.5]$; lower values sharpen the mode, higher values promote structural diversity.

**Top-$k$ filtering** (default $k$=50) discards all but the $k$ most probable tokens, removing low-probability candidates that frequently yield invalid SMILES.

**Top-$p$ (nucleus) sampling** [11] dynamically selects the smallest token set whose cumulative probability exceeds $p$=0.95, adapting the effective vocabulary to the model's per-step confidence.

**Repetition penalty** ($\vartheta$=1.15) down-scales logits of previously emitted tokens:

$$\hat{l}_{t,k} = l_{t,k}/\vartheta \quad \text{if } k \in \text{prev. tokens and } l_{t,k} > 0 \tag{7}$$

Additionally, a `min_new_tokens = 8` guard suppresses `<EOS>` during the first eight steps to prevent degenerate single-atom outputs, and a `max_repeat_run = 4` detector blocks any token that has appeared four times consecutively.

## VII. CHEMICAL EVALUATION

### A. Property Computation

Every generated SMILES is evaluated by an RDKit-based pipeline (`src/property.py`) that computes nine molecular descriptors (Table III). A composite Lipinski Ro5 flag is derived separately: it requires simultaneous satisfaction of four conditions (MW $\leq$ 500, LogP $\leq$ 5, HBD $\leq$ 5, HBA $\leq$ 10).

### B. Sampling-Control Ablation

Table IV reports a cumulative ablation of the decoding controls ($T$=0.8, seed = `"C"`, $n$=500 molecules per condition).

The full pipeline raises validity by **6.2 percentage points** and mean QED by 0.134 (22% relative) compared with temperature-only decoding, confirming that each control contributes measurably.

TABLE III
MOLECULAR DESCRIPTORS AND THEIR DRUG-LIKENESS RANGES.

| Property | RDKit function | Drug-like range |
|---|---|---|
| Mol. weight (MW) | `Descriptors.MolWt` | 150–500 Da |
| LogP | `Descriptors.MolLogP` | $-1$ to 5 |
| QED | `QED.qed` | $> 0.5$ |
| SA score | `sascorer.calculateScore` | 1–4 |
| TPSA | `Descriptors.TPSA` | <140 $A^{\circ 2}$ |
| HBD | `Descriptors.NumHDonors` | $\leq 5$ |
| HBA | `Descriptors.NumHAcceptors` | $\leq 10$ |
| Rot. bonds | `Descriptors.NumRotatableBonds` | $\leq 10$ |
| Ring count | `Descriptors.RingCount` | $\geq 1$ |
| *Lipinski Ro5* | *Derived composite flag* | *All 4 above* |

TABLE IV
CUMULATIVE ABLATION OF GENERATION CONTROLS AT $T$=0.8 ($n$=500, SEED = "C").

| Configuration | Validity | Avg QED | Uniqueness |
|---|---|---|---|
| Temperature only ($T$=0.8) | 77.8% | 0.61 | 94.1% |
| + Top-$k$ ($k$=50) | 80.3% | 0.65 | 95.7% |
| + Top-$p$ ($p$=0.95) | 82.1% | 0.68 | 96.2% |
| + Rep. penalty ($\vartheta$=1.15) | 83.6% | 0.72 | 97.1% |
| **Full pipeline (default)** | **84.0%** | **0.744** | **97.8%** |

TABLE V
GENERATION QUALITY ACROSS TEMPERATURES ($n$=500, SEEDS "C", "N", "c1ccccc1" AVERAGED).

| $T$ | Validity | QED | SA | Lipinski | IntDiv |
|---|---|---|---|---|---|
| 0.5 | 91.2% | 0.54 | 3.2 | 82.0% | 0.841 |
| 0.8 | 84.0% | 0.744 | 2.99 | 96.4% | 0.853 |
| 1.0 | 77.3% | 0.61 | 3.8 | 71.0% | 0.861 |
| 1.2 | 69.1% | 0.63 | 4.1 | 65.0% | 0.868 |

TABLE VI
COMPARISON ON ZINC-250K. † MOSES BENCHMARK [13]; ‡BAGAL ET AL. [17]. INTDIV: MEAN PAIRWISE TANIMOTO DISTANCE.

| Model | Valid | Unique | Novel | IntDiv |
|---|---|---|---|---|
| CharRNN† [12] | 97.4% | 99.8% | 84.5% | 0.856 |
| VAE† [14] | 67.5% | 99.8% | 68.9% | 0.856 |
| MolGPT‡ [17] | 88.9% | 96.5% | 94.0% | 0.854 |
| **Ours** ($T$=0.8) | **84.0%** | **97.8%** | **99.2%** | **0.853** |

### C. Temperature Sensitivity

Table V examines generation quality at four temperature settings ($n$=500 per condition, three seed fragments averaged). Validity is the fraction of SMILES for which `Chem.MolFromSmiles` returns a valid molecule object. Internal diversity (IntDiv) is the mean pairwise Tanimoto distance over Morgan fingerprints within each batch.

A clear validity–diversity trade-off is evident: $T$=0.5 maximises validity (91.2%) at the cost of diversity, while higher temperatures increase IntDiv but degrade both validity and Lipinski compliance. $T$=0.8 offers a practical balance—84.0% validity, QED 0.744, SA 2.99, 96.4% Lipinski, IntDiv 0.853—and is used as the default operating point. All metrics in Tables V and VI are reported as means over three seed fragments (`"C"`, `"N"`, `"c1ccccc1"`); because generation is stochastic, individual runs may vary by approximately $\pm$1–2%.

### D. Baseline Comparison

Table VI positions MolecularTransformer against published baselines on ZINC-250k. CharRNN and VAE figures are drawn from the MOSES benchmark [13]; MolGPT figures are from Bagal et al. [17]. Uniqueness is measured as the fraction of distinct canonical SMILES among valid outputs; novelty is the fraction of valid molecules whose canonical SMILES do not appear in the 199,564-molecule training split.

MolecularTransformer achieves validity and IntDiv competitive with MolGPT while using only 3.2 M parameters—roughly half the size typical of comparable SMILES generators (4–6 M). The high novelty (99.2%) reflects the generative diversity of temperature-based sampling on a compact model that has not memorised the training corpus. This compact footprint is a deliberate design choice enabling training on free-tier T4 GPU instances.

## VIII. STREAMLIT DEPLOYMENT

### A. Application Architecture

The trained model is deployed as a Streamlit web application that requires no local installation. Users select a checkpoint, specify a seed fragment and sampling parameters, and receive results organised into three panels: 3D visualisation, chemical-space analysis, and batch analytics.

### B. 3D Molecular Visualisation

Valid molecules are embedded in three-dimensional coordinates via RDKit's `AllChem.EmbedMolecule` (seed 42) with MMFF force-field optimisation (`AllChem.MMFFOptimizeMolecule`, 200 iterations). The resulting `.mol` block is rendered as an interactive ball-and-stick model using py3Dmol, streamed into the browser through `components.html`.

### C. Chemical-Space Projection (t-SNE)

Generated and reference molecules are represented as Morgan fingerprints (radius 2, 2048 bits) and projected to two dimensions with t-SNE [9] (perplexity 5). An interactive Plotly scatter plot colours generated molecules red and known drugs grey, allowing visual assessment of distributional overlap. Hover tooltips display SMILES, MW, and LogP.

### D. Drug Similarity Search

Each generated molecule is compared to a curated reference set (`data/known_drugs.csv`) via Tanimoto similarity on Morgan fingerprints. The top-3 most similar approved drugs are reported with similarity scores and therapeutic categories.

### E. Batch Analytics

Aggregate statistics—validity rate, mean pairwise Tanimoto distance, average QED, average MW, average LogP, and Lipinski pass rate—are computed over the full generated batch and displayed as Plotly histograms and summary metric cards.

## IX. Limitations and Future Work

Several limitations warrant discussion. The model generates molecules purely from learned syntax and is not conditioned on biological targets, binding affinities, or assay data. Chemical validity is checked post-generation rather than enforced structurally, so invalid SMILES can be produced before RDKit filtering. The linearised SMILES representation does not capture 3D geometry or stereochemistry directly. Finally, the compact 3.2 M-parameter scale limits coverage of chemical space relative to larger pre-trained systems [21].

Promising extensions include: (i) reinforcement-learning fine-tuning with QED or docking-score rewards [24] for property-conditioned generation; (ii) SELFIES-based decoding for guaranteed syntactic validity; (iii) diffusion-based 3D generation [20] to model spatial geometry; (iv) scaling $d_{\text{model}}$ and pre-training on the full billion-molecule ZINC corpus; and (v) complete MOSES/GuacaMol evaluation including FCD.

## X. Conclusion

We presented MolecularTransformer, a 3.2 M-parameter causal Transformer for de novo small-molecule generation trained on ZINC-250k. The system provides an integrated, fully open-source pipeline covering scaffold-aware data splitting, regex-based SMILES tokenisation, multi-stage generation with validated sampling controls, nine-descriptor RDKit evaluation, and interactive Streamlit deployment.

At the default operating point ($T = 0.8$), MolecularTransformer achieves 84.0% chemical validity, mean QED of 0.744, mean SA score of 2.99, 96.4% Lipinski compliance, and internal diversity of 0.853—competitive with MolGPT [17] at roughly half the parameter count. Ablation confirms that each stage of the decoding pipeline contributes measurably, yielding a cumulative 6.2-percentage-point improvement in validity over temperature-only sampling.

By demonstrating that a deliberately compact Transformer can produce high-quality, drug-like molecules on free-tier GPU infrastructure, this work aims to make AI-assisted molecular design accessible to researchers without specialised hardware. The complete codebase, trained checkpoint, and live web application are publicly available.

## Acknowledgment

The authors thank Dr. Monika for academic supervision and IILM University for institutional support. Experiments were conducted on Google Colab with NVIDIA T4 GPU infrastructure.

## References

[1] P. G. Polishchuk, T. I. Madzhidov, and A. Varnek, "Estimation of the size of drug-like chemical space based on GDB-17 data," *J. Comput.-Aided Mol. Des.*, vol. 27, no. 8, pp. 675–679, 2013.

[2] J. A. DiMasi, H. G. Grabowski, and R. W. Hansen, "Innovation in the pharmaceutical industry: New estimates of R&D costs," *J. Health Econ.*, vol. 47, pp. 20–33, 2016.

[3] D. Weininger, "SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules," *J. Chem. Inf. Comput. Sci.*, vol. 28, no. 1, pp. 31–36, 1988.

[4] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[5] G. W. Bemis and M. A. Murcko, "The properties of known drugs. 1. Molecular frameworks," *J. Med. Chem.*, vol. 39, no. 15, pp. 2887–2893, 1996.

[6] G. R. Bickerton, G. V. Paolini, J. Besnard, S. Muresan, and A. L. Hopkins, "Quantifying the chemical beauty of drugs," *Nature Chem.*, vol. 4, no. 2, pp. 90–98, 2012.

[7] P. Ertl and A. Schuffenhauer, "Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions," *J. Cheminf.*, vol. 1, no. 1, p. 8, 2009.

[8] C. A. Lipinski, F. Lombardo, B. W. Dominy, and P. J. Feeney, "Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings," *Adv. Drug Deliv. Rev.*, vol. 46, no. 1–3, pp. 3–26, 2001.

[9] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[10] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019.

[11] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2020.

[12] M. H. S. Segler, T. Kogej, C. Tyrchan, and M. P. Waller, "Generating focused molecule libraries for drug discovery with recurrent neural networks," *ACS Central Sci.*, vol. 4, no. 1, pp. 120–131, 2018.

[13] D. Polykovskiy *et al.*, "Molecular sets (MOSES): A benchmarking platform for molecular generation models," *Front. Pharmacol.*, vol. 11, p. 565644, 2020.

[14] R. Go´mez-Bombarelli *et al.*, "Automatic chemical design using a data-driven continuous representation of molecules," *ACS Central Sci.*, vol. 4, no. 2, pp. 268–276, 2018.

[15] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2014.

[16] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," in *Proc. Conf. Computational Natural Language Learning (CoNLL)*, pp. 10–21, 2016.

[17] V. Bagal, R. Aggarwal, P. K. Vinod, and U. D. Priyakumar, "MolGPT: Molecular generation using a transformer-decoder model," *J. Chem. Inf. Model.*, vol. 62, no. 9, pp. 2064–2076, 2022.

[18] S. Chithrananda, G. Grand, and B. Ramsundar, "ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction," *arXiv:2010.09885*, 2020.

[19] M. Krenn, F. Ha¨se, A. Nigam, P. Friederich, and A. Aspuru-Guzik, "Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation," *Mach. Learn.: Sci. Technol.*, vol. 1, no. 4, p. 045024, 2020.

[20] E. Hoogeboom, V. G. Satorras, C. Vignac, and M. Welling, "Equivariant diffusion for molecule generation in 3D," in *Proc. Int. Conf. Machine Learning (ICML)*, vol. 162, pp. 8867–8887, 2022.

[21] R. Irwin, S. Dimitriadis, J. He, and E. J. Bjerrum, "Chemformer: A pre-trained transformer for computational chemistry," *Mach. Learn.: Sci. Technol.*, vol. 3, no. 1, p. 015022, 2022.

[22] J. J. Irwin, T. Sterling, M. M. Mysinger, E. S. Bolstad, and R. G. Coleman, "ZINC: A free tool to discover chemistry for biology," *J. Chem. Inf. Model.*, vol. 52, no. 7, pp. 1757–1768, 2012.

[23] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in *Proc. SPIE*, vol. 11006, 2019.

[24] M. Olivecrona, T. Blaschke, O. Engkvist, and H. Chen, "Molecular de-novo design through deep reinforcement learning," *J. Cheminf.*, vol. 9, no. 1, p. 48, 2017.